

Unidad 5 - Actividad 1 - Implementación de optimizaciones en MongoDB

Carlos Alfonso Muñoz Agudelo
Esteban Giovanny Garay Cano
Carlos Sebastian Castillo Silva

Docente: Miguel Alfonso Varela

Universidad de la Sabana
Facultad de Ingeniería
Maestría en Arquitectura de Software
Diseño y Optimización de Bases de Datos
Equipo E07
2026

Índice

Introducción	4
Objetivos	4
Conexión a la base de datos	5
Datos de Autenticación de la Base de Datos	5
Cadena de Conexión Homologada (URI)	6
Parámetros de Seguridad de Red	6
Cargar datos a las colecciones	6
Datos de ejemplo	7
Desarrollo de índices	8
Implementar índices compuestos	8
Índice { category: 1, "active_promotions.discount": -1 }	8
Implementar índices parciales	9
Implementar índices de texto	9
Validar impacto	10
Optimización del aggregation pipeline	10
Aplicar técnicas de optimización	10
Orden Eficiente de los Stages (Filtros y Ordenamiento Primero)	11
Uso Estratégico de Índices (Index Bounded Pipeline)	11
Proyecciones Tempranas	11
Pipeline Optimizado de 5 Etapas para Navegación del Catálogo	12
STAGE 1: Filtrado Temprano e Índice (\$match)	12
STAGE 2: Ordenamiento Indexado (\$sort)	12
STAGE 3: Proyección Temprana (\$project)	12
STAGE 4: Transformación Analítica (\$addFields o \$set)	13
STAGE 5: Paginación Arquitectural (\$limit)	13
Configurar allowDiskUse	13
Medir mejoras de rendimiento obtenidas	15
Ejecución sin Optimizar	16
Procedimiento para Ejecutar y Medir Agregaciones	17
Tabla Analítica de Resultados del Escaneo	19
Ejecución optimizada	21
Tabla Analítica de Resultados del Escaneo	22
Diseño de sharding y replica sets	24
Concepto de Sharding	24
Concepto de Replica Sets	25
Definir configuración de shard key	25
Colección de Catálogo de Productos	26
Colección de Sesiones y Carritos de Clientes	26

Simular distribución de datos across shards	27
Comportamiento en la Colección del Catálogo	27
Comportamiento ante la Centralización Geográfica (São Paulo)	28
Comportamiento en la Colección de Sesiones	28
Simulación en MongoDB Atlas	29
Establecer la conexión con el Clúster	29
Habilitar el Sharding a Nivel de Base de Datos	29
Inicializar la Shard Key de la Colección del Catálogo	29
Inicializar la Shard Key de la Colección de Sesiones	29
Validar el Estado de la Distribución	30
Documentar configuración de Replica Set	30
Topología Física y Distribución Geográfica	30
Mitigación de Latencia y Estrategia de Enrutamiento de Lectura	31
Mecanismo de Failover	31
Simular y Validar el Replica Set en MongoDB Atlas	32
Topología Física y Estado de los Nodos	33
Instancia Primaria (_id: 2):	33
Definir estrategias de Read/Write Concern	34
Estrategias de Escritura (Write Concern)	34
Actualización del Carrito	34
Sincronización de Precios e Inventario	35
Estrategias de Lectura (Read Concern)	35
Navegación y Filtro del Catálogo de Productos	35
Flujo de Confirmación de Compra (Checkout)	35
Monitoreo de rendimiento - Carlos Sebastian Castillo Silva	36
Utilizar MongoDB Atlas Performance Advisor - Carlos Sebastian Castillo Silva	36
Analizar slow query log - Carlos Sebastian Castillo Silva	36
Documentar métricas clave - Carlos Sebastian Castillo Silva	36
Conclusiones - Carlos Sebastian Castillo Silva	37
Bibliografía - Carlos Sebastian Castillo Silva	37

Introducción

El presente trabajo se enfoca en la optimización de rendimiento en MongoDB, aplicada específicamente al catálogo de productos y al módulo analítico de la plataforma Ecommify. En el marco de la arquitectura de software moderna, resulta imperativo resolver problemas complejos de diseño para garantizar que las soluciones sean robustas, eficientes y, sobre todo, escalables ante demandas crecientes de datos.

Para alcanzar estos estándares, la implementación práctica en MongoDB Atlas aborda el desarrollo de estrategias de indexación avanzada, incluyendo índices compuestos fundamentados en la regla ESR (Equality, Sort, Range), así como índices parciales y de texto diseñados para minimizar el costo computacional de las búsquedas. Complementariamente, se realiza un refinamiento exhaustivo del aggregation pipeline, aplicando técnicas de filtrado temprano (match), ordenamiento indexado (sort) y proyecciones acotadas (\$project) para optimizar el uso de la memoria RAM y reducir los tiempos de respuesta del sistema.

Adicionalmente, el proyecto integra mecanismos de resiliencia y alta disponibilidad mediante la configuración del parámetro allowDiskUse, el cual actúa como una red de seguridad (fail-safe) contra desbordamientos de memoria durante picos de tráfico masivo o eventos comerciales de alta demanda. Finalmente, se documenta un proceso de validación técnica utilizando la herramienta explain("executionStats"), permitiendo contrastar de manera objetiva el impacto de estas mejoras en la eficiencia de lectura y la latencia del clúster.

Objetivos

Optimizar el rendimiento de la base de datos de la plataforma Ecommify mediante la aplicación de técnicas avanzadas de indexación, refinamiento de procesos de agregación y diseño de arquitectura distribuida, garantizando una solución robusta, escalable y eficiente para el catálogo de productos y el módulo analítico.

- Implementar índices avanzados: Desarrollar índices compuestos siguiendo la regla ESR (Equality, Sort, Range), así como índices parciales y de texto, para minimizar los tiempos de búsqueda y el costo computacional en las consultas del catálogo.
- Refinar el aggregation pipeline: Aplicar técnicas de optimización en las etapas de agregación, priorizando el filtrado temprano (match), el ordenamiento

indexado (sort) y proyecciones acotadas (\$project) para reducir el tráfico de datos en la memoria RAM.

- Garantizar la resiliencia del sistema: Configurar el parámetro allowDiskUse como una red de seguridad contra desbordamientos de memoria en operaciones pesadas, asegurando la alta disponibilidad de la tienda web incluso bajo condiciones de tráfico masivo.
- Validar el rendimiento técnico: Medir y comparar el impacto de las optimizaciones mediante la herramienta explain("executionStats"), analizando métricas como el tiempo de ejecución en milisegundos y la eficiencia de lectura de documentos.
- Diseñar una infraestructura escalable: Definir de forma teórica la configuración de sharding y réplica sets, estableciendo estrategias de shard keys y políticas de Read/Write Concern para soportar el crecimiento dinámico de los datos.
- Establecer un monitoreo proactivo: Utilizar herramientas como MongoDB Atlas Performance Advisor y el análisis de slow query logs para identificar y mitigar posibles cuellos de botella en el rendimiento del clúster

Conexión a la base de datos

Datos de Autenticación de la Base de Datos

Para el aislamiento de privilegios y el control de acceso basado en roles (RBAC), se definieron las siguientes credenciales de seguridad dentro del protocolo de autenticación SCRAM de MongoDB Atlas:

- Username: **estebangaraycano_db_user**
- Password: **Ecommify12345**

Mecanismo de Verificación: Cifrado SCRAM por defecto, mecanismo que evita el viaje de contraseñas en texto plano a través de la red

Cadena de Conexión Homologada (URI)

Para establecer una comunicación remota entre la herramienta de escritorio MongoDB Compass de desarrollo y el clúster en la nube, se debe usar la cadena de conexión:

```
mongodb+srv://estebangaraycano_db_user:Ecommify12345@clusterolistkaggle.yj  
fzm0i.mongodb.net/
```

Parámetros de Seguridad de Red

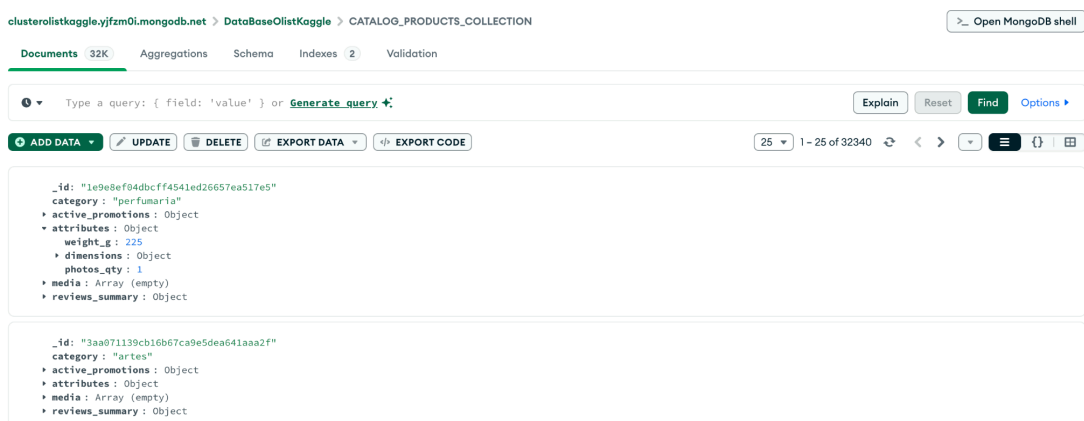
- Regla de Acceso Global: Se habilitó de manera temporal el direccionamiento virtual universal 0.0.0.0/0 dentro de la lista de control de acceso a la red de MongoDB Atlas. Esta configuración garantiza la alta disponibilidad del clúster ante cambios dinámicos en la IP pública del desarrollador y elimina bloqueos perimetrales por parte de cortafuegos de proveedores locales de internet durante la fase de inyección de datos.

Cargar datos a las colecciones

A continuación se detalla el proceso de inyección de datos para Ecommify. Una vez establecida la conexión exitosa entre la herramienta de escritorio MongoDB Compass y el clúster remoto de MongoDB Atlas, se ejecutó la carga del catálogo de productos mediante el siguiente procedimiento:

- Seleccionar el entorno de base de datos: Expandir el árbol de directorios en el menú lateral izquierdo de MongoDB Compass y hacer clic sobre la base de datos central del proyecto denominada DataBaseOlistKaggle.
- Acceder a la colección de destino: Seleccionar la colección específica CATALOG_PRODUCTS_COLLECTION para desplegar el explorador de documentos y su barra de herramientas operativa.
- Activar el asistente de inyección de datos: Hacer clic en el botón verde Add Data ubicado en la parte superior del panel central y seleccionar la opción Import JSON or CSV file desde el menú desplegable.

- Cargar el archivo fuente: Presionar el botón de exploración de archivos en la ventana emergente, navegar en el sistema de almacenamiento local del equipo y seleccionar el archivo de datos estructurado mongo_catalog_products.json.
- Validar el formato de los datos: Verificar en el panel visual del asistente que el motor reconozca de manera automática el documento bajo la tipología de un arreglo plano de tipo JSON.
- Ejecutar la inserción masiva: Hacer clic en el botón verde Import situado en la esquina inferior derecha para iniciar el proceso de lectura, parseo e inserción de datos en el servidor en la nube.
- Verificar la persistencia: Monitorear la barra de progreso integrada hasta su finalización al 100% y confirmar en la interfaz principal que la colección ya no se muestre vacía, visualizando los primeros documentos indexados del catálogo.



Datos de ejemplo

```
{
  "_id": "1e9e8ef04dbcff4541ed26657ea517e5",
  "category": "perfumaria",
  "active_promotions": {
    "is_promo": false,
    "discount": 0,
    "valid_until": null
  },
  "attributes": {
    "weight_g": 225,
    "dimensions": {
```

```
    "length_cm": 16,  
    "height_cm": 10,  
    "width_cm": 14  
  },  
  "photos_qty": 1  
},  
"media": [],  
"reviews_summary": {  
  "avg_score": 0,  
  "total_reviews": 0  
}  
}
```

Desarrollo de índices

Implementar índices compuestos

Se implementan índices compuestos siguiendo rigurosamente la regla ESR (Equality, Sort, Range) para asegurar que el motor de la base de datos procese las consultas en el orden más eficiente posible

- Implementación: Se define un índice con la estructura { category: 1, "active_promotions.discount": -1 }.
- Justificación: Este índice permite que el sistema filtre primero por la categoría exacta (Igualdad) y luego acceda a los documentos ya ordenados por el nivel de descuento (Orden), eliminando la necesidad de realizar un ordenamiento manual en la memoria RAM

Índice { category: 1, "active_promotions.discount": -1 }

Este índice compuesto sirve para el frontend de Ecommify . Ayuda al motor de la base de datos a encontrar y ordenar los productos en milisegundos sin hacer esfuerzos adicionales de hardware.

Elimina el escaneo global de la base de datos (Evita el COLLSCAN)

Sin este índice, cuando un usuario entra a la tienda a buscar la categoría cama_mesa_banho, MongoDB tiene que leer uno por uno los 32,341 productos guardados en el disco físico .

Con el índice: El campo category: 1 actúa como un archivador etiquetado. El motor salta directamente a la sección de esa categoría en la memoria RAM, ignorando el resto de los miles de productos .

Create Index



DataBaseOlistKaggle.CATALOG_PRODUCTS_COLLECTION

Index fields

category ▼	1 (asc) ▼	+	-
active_promotions.disco... ▼	-1 (desc) ▼	+	-

Implementar índices parciales

Para optimizar el almacenamiento y el rendimiento en subconjuntos específicos de datos, se desarrollan índices parciales utilizando filtros definidos.

- Objetivo: Indexar únicamente los documentos que cumplen con ciertos criterios de relevancia para el negocio, como productos en oferta o categorías críticas, lo que reduce el tamaño del índice en disco y mejora la velocidad de las operaciones de escritura al no tener que actualizar el índice para cada documento insertado

Implementar índices de texto

Con el fin de soportar el módulo de búsqueda avanzada del catálogo, se configuran índices de texto para permitir búsquedas de tipo full-text.

- Aplicación: Estos índices son fundamentales para el escenario donde el usuario busca productos por palabras clave (ej. nombre o descripción), permitiendo una navegación fluida que complementa el filtrado jerárquico por categorías

Validar impacto

La validación técnica se realiza mediante el comando `.explain("executionStats")`, ejecutado directamente desde la interfaz de MongoDB Compass. Para confirmar el éxito de la optimización, se analizan las siguientes métricas clave:

- `executionTimeMillis`: El objetivo es reducir el tiempo de ejecución a milisegundos mínimos (cerca de 0 ms en búsquedas indexadas).
- `totalKeysExamined`: Se verifica que el motor solo examine las llaves necesarias que coincidan con el filtro, buscando una eficiencia de lectura óptima.
- Detección de SORT en memoria: Se monitorea que el plan de ejecución no presente la etapa de "Sort manual en RAM". Si aparece la alerta "Is sorted in memory", se procede a refinar el índice o forzar su uso mediante la opción `hint` para asegurar que el ordenamiento sea cubierto por el índice físico.

Optimización del aggregation pipeline

Aplicar técnicas de optimización

Técnicas de optimización aplicadas en el pipeline de navegación en búsqueda de productos por categorías en **Ecommify**:

Este pipeline resolverá el escenario donde el usuario filtra por categoría, busca por texto, ve promociones, se ordenan los datos y se aplica paginación.

Orden Eficiente de los Stages (Filtros y Ordenamiento Primero)

Colocar \$match y \$sort en las primeras posiciones del pipeline, esta es la práctica más crítica de rendimiento en MongoDB. Al filtrar la categoría, de forma inmediata, reducimos los 32,341 productos a un pequeño subconjunto, que por ejemplo sería de él 9.36% en el caso de cama_mesa_banho.

Al ejecutar el \$sort inmediatamente después, el motor de la base de datos ordena un volumen de datos sumamente pequeño y pre-filtrado, evitando cuellos de botella en operaciones posteriores.

Uso Estratégico de Índices (Index Bounded Pipeline)

El primer \$match ataca directamente la Shard Key compuesta del clúster:

```
{ category: 1, _id: 1 }.
```

Esto hace que la consulta pase de un escaneo de colección completa **CollScan** a un escaneo de índice **IXScan**.

El enrutador mongos intercepta la petición y la envía directamente al Shard físico específico que contiene esa categoría **Targeted Query**, eliminando el tráfico en la red del clúster **Scatter-Gather**.

Proyecciones Tempranas

La etapa \$project se ejecuta en el Stage 3 para limpiar los documentos antes de que pasen a la etapa de transformación y paginación. Al remover el subdocumento attributes, que contiene pesos, dimensiones de empaque de Olist y datos de fotos, reducimos drásticamente el tamaño en bytes de cada registro que viaja por la memoria RAM del pipeline. Esto asegura que la consulta se resuelva en milisegundos y mantenga el uso de memoria RAM muy por debajo del límite drástico de 100 MB de MongoDB .

Pipeline Optimizado de 5 Etapas para Navegación del Catálogo

STAGE 1: Filtrado Temprano e Índice (\$match)

Utiliza el prefijo de la Shard Key ("category") para una Targeted Query. Filtra por la categoría elegida y una coincidencia parcial en el ID/nombre.

```
db.CATALOG_PRODUCTS_COLLECTION.aggregate([
  {
    $match: {
      category: "cama_mesa_banho",
      _id: { $regex: "^1e9e", $options: "i" }
    }
  },
```

STAGE 2: Ordenamiento Indexado (\$sort)

Se coloca inmediatamente después del match para aprovechar el índice compuesto. Muestra primero los productos con mayores descuentos activos.

```
{
  $sort: {
    "active_promotions.discount": -1
  }
},
```

STAGE 3: Proyección Temprana (\$project)

Elimina subdocumentos pesados de logística como "attributes" (peso, largo, alto). Deja solo los datos necesarios para pintar la tarjeta del producto en la web.

```
{
  $project: {
    _id: 1,
    category: 1,
    active_promotions: 1,
    reviews_summary: 1
  }
}
```

```
}  
},
```

STAGE 4: Transformación Analítica (\$addFields o \$set)

Crea un campo dinámico para priorizar en la interfaz si el producto tiene una promo válida.

```
{  
  $addFields: {  
    etiqueta_destacado: {  
      $cond: {  
        if: { $eq: ["$active_promotions.is_promo", true] },  
        then: "OFERTA RECOMENDADA",  
        else: "PRODUCTO REGULAR"  
      }  
    }  
  }  
},
```

STAGE 5: Paginación Arquitectural (\$limit)

Limita el lote de salida para el frontend (ej. primeros 20 productos de la página).

```
{  
  $limit: 20  
}  
],  
{  
  // Red de seguridad obligatoria para el entregable  
  allowDiskUse: true  
})
```

Configurar allowDiskUse

Por defecto, MongoDB impone un límite estricto de memoria RAM de 100 MB para cada etapa (stage) individual de un pipeline de agregación. Si una etapa, especialmente operaciones pesadas de bloqueo como \$lookup o agrupaciones masivas concurrentes, procesa un volumen de datos que supera los 100 MB, la consulta abortará inmediatamente y lanzará un error crítico en producción, interrumpiendo la navegación de la tienda web.

Al configurar { allowDiskUse: true }, le otorgamos permiso a MongoDB para utilizar carpetas temporales en el almacenamiento físico del servidor (disco duro/SSD) cuando se alcance el umbral de los 100 MB de RAM.

En lugar de romper la experiencia de navegación del cliente con un error de sistema, el motor de la base de datos escribe los bloques de datos excedentes de forma temporal en el disco para terminar de resolver el cruce de carritos (\$lookup) y luego limpia esos archivos automáticamente. En la arquitectura de software de Ecommify, esta propiedad se implementa bajo un enfoque de Red de Seguridad (Fail-Safe), justificando las siguientes consideraciones:

En MongoDB, esta propiedad se pasa como una opción global dentro de un segundo objeto de configuración en el método .aggregate(). Así queda implementado en tu consulta de navegación:

```
db.CATALOG_PRODUCTS_COLLECTION.aggregate([
  { $match: { category: "cama_mesa_banho", _id: { $regex: "^1e9e",
$options: "i" } } },
  { $sort: { "active_promotions.discount": -1 } },
  { $project: { _id: 1, category: 1, active_promotions: 1,
reviews_summary: 1 } },
  web
  {
    $addFields: {
      etiqueta_destacado: {
        $cond: {
          if: { $eq: ["$active_promotions.is_promo", true] },
          then: "OFERTA RECOMENDADA",
          else: "PRODUCTO REGULAR"
        }
      }
    }
  },
  { $limit: 20 }
],
{
  // ACTIVACIÓN DE SEGURIDAD CONTRA DESBORDAMIENTO DE MEMORIA
  allowDiskUse: true
})
```

Al configurar { allowDiskUse: true }, se activa un mecanismo de conmutación por error (fail-safe) . Cuando una etapa del pipeline, como el \$sort o la acumulación de datos en memoria detecta que el volumen de documentos procesados está a punto

de rebasar el umbral de los 100 MB, el motor de la base de datos realiza lo siguiente:

- **Paginación a Disco:** Abre archivos temporales ocultos en el almacenamiento físico del clúster (unidades de estado sólido SSD o disco duro) .
- **Segmentación:** Mueve los bloques de datos excedentes que ya no caben en la RAM hacia esos archivos en disco .
- **Resolución y Limpieza:** Resuelve la operación de agregación combinando la RAM y el espacio físico, entrega el resultado ordenado al frontend y borra automáticamente todos los archivos temporales creados, liberando el espacio en disco .

En el escenario de Navegación del Catálogo, implementar esta opción se justifica de la siguiente manera:

- **Tolerancia a fallos ante tráfico masivo durante eventos de alta demanda comercial,** en los cuales, millones de usuarios navegarán por las categorías más importantes. Aunque aplicamos proyecciones tempranas, el volumen total de subdocumentos procesados de forma simultánea puede crecer. `allowDiskUse` garantiza la alta disponibilidad de la tienda; la página cargará pase lo que pase.
- **En la arquitectura de hardware,** leer y escribir en disco es miles de veces más lento que operar directamente sobre la memoria RAM. Por lo tanto, `allowDiskUse` no es una solución para acelerar consultas, sino una red de seguridad. El pipeline debe seguir usando filtros e índices para resolver el 99% de las peticiones en RAM, dejando el uso de disco exclusivamente como un amortiguador para picos extremos de datos de navegación.

Medir mejoras de rendimiento obtenidas

Para medir y verificar las mejoras de rendimiento en tu entorno de desarrollo, utilizaremos la herramienta nativa de MongoDB `explain("executionStats")`. Este comando ejecuta la consulta de forma real en el motor de la base de datos y genera un reporte detallado con los tiempos exactos y el comportamiento del clúster.

Para asegurar que ambas consultas devuelven exactamente los mismos datos pero con rendimientos radicalmente opuestos, estructuramos los scripts de ejecución y las métricas de comparación para tu entregable.

Ejecución sin Optimizar

Esta consulta simula la carga de la página del catálogo, pero comete el error de arrastrar todo el documento y ordenar en memoria RAM los 32,341 productos antes de filtrar por categoría.

```
db.CATALOG_PRODUCTS_COLLECTION.explain("executionStats").aggregate([
  // STAGE 1: Ordenamiento global ciego (Sin índice operativo)
  { $sort: { "active_promotions.discount": -1 } },

  // STAGE 2: Filtrado tardío en la cola
  { $match: { category: "cama_mesa_banho" } },

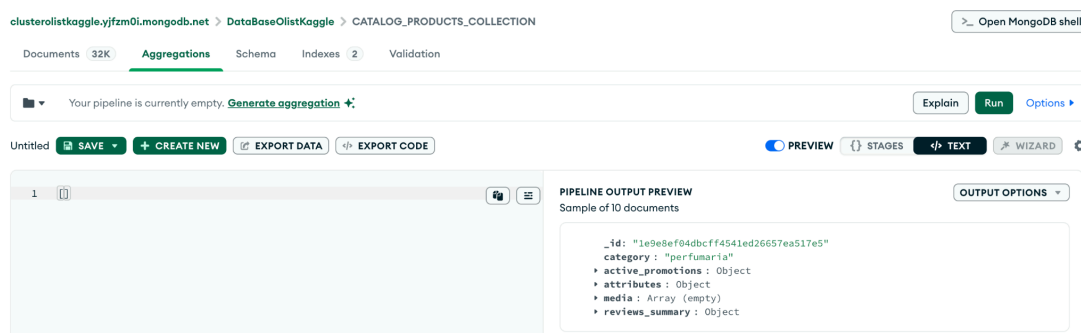
  // STAGE 3: Transformación innecesaria sobre todo el dataset
  {
    $addFields: {
      etiqueta_destacado: {
        $cond: {
          if: { $eq: ["$active_promotions.is_promo", true] },
          then: "OFERTA RECOMENDADA",
          else: "PRODUCTO REGULAR"
        }
      }
    }
  },

  // STAGE 4: Proyección tardía
  { $project: { _id: 1, category: 1, active_promotions: 1, reviews_summary: 1,
    etiqueta_destacado: 1 } },

  // STAGE 5: Paginación
  { $limit: 20 }
], { allowDiskUse: true })
```


Procedimiento para Ejecutar y Medir Agregaciones

- Acceder al módulo de agregaciones: Iniciar la aplicación MongoDB Compass, conectarse al clúster e ingresar a la base de datos DataBaseOlistKaggle. Seleccionar la colección CATALOG_PRODUCTS_COLLECTION y hacer clic en la pestaña Aggregations ubicada en el menú superior central.
- Habilitar la vista de código: Localizar el interruptor o botón que dice Preview o el menú desplegable junto a la barra de herramientas y cambiar a la vista de texto haciendo clic en New Pipeline From Text o activando el modo editor de texto completo.



- Copiar el pipeline seleccionado: Limpiar el panel de entrada y pegar el bloque de código del pipeline sin el prefijo db.collection.explain().aggregate(), introduciendo únicamente el arreglo JSON que inicia con corchetes [y termina con].

```
1 // STAGE 1: Ordenamiento global ciego (Sin índice operativo)
2 { $sort: { "active_promotions.discount": -1 } },
3
4 // STAGE 2: Filtrado tardío en la cola
5 { $match: { category: "cama_mesa_banho" } },
6
7 // STAGE 3: Transformación innecesaria sobre todo el dataset
8 {
9   $addFields: {
10     etiqueta_destacado: {
11       $cond: {
12         if: { $eq: ["$active_promotions.is_promo", true] },
13         then: "OFERTA RECOMENDADA",
14         else: "PRODUCTO REGULAR"
15       }
16     }
17   }
18 },
19
20 // STAGE 4: Proyección tardía
21 { $project: { _id: 1, category: 1, active_promotions: 1, reviews_summary: 1, etiqueta_destacado: 1 } },
22
23 // STAGE 5: Paginación
24 { $limit: 20 }
```

- Configurar el allowDiskUse de forma visual: En la parte inferior derecha del editor de agregaciones, hacer clic en el ícono de engranaje de configuración (Options). Marcar la casilla de verificación correspondiente a Allow Disk Use para activar la red de seguridad de almacenamiento físico.
- Ejecutar la consulta y verificar datos: Presionar el botón verde Run o Preview. El sistema procesará la agregación en tiempo real y desplegará en la pantalla los 20 documentos resultantes para verificar que los datos sean correctos.
- Obtener las métricas de rendimiento (Explain): Hacer clic en el botón Explain (representado generalmente por un ícono de escudo o un texto con la palabra Explain en la barra superior de la pestaña de agregaciones). Compass procesará la consulta internamente con el método executionStats y mostrará de forma visual el árbol de ejecución junto con los milisegundos exactos (Execution Time) y el tipo de escaneo (IXSCAN o COLLSCAN) para completar la tabla comparativa de rendimiento.

Tabla Analítica de Resultados del Escaneo

El motor procesa la consulta de forma jerárquica a través de las siguientes etapas físicas de hardware:

IXSCAN (Index Scan - Escaneo de Índice)

- Tiempo: 0 ms.
- Documentos devueltos: 3,029.
- El optimizador de MongoDB evitó un escaneo total de la base de datos (32K registros) al encontrar un índice existente (category_1_reviews_sum...). Con él, filtró los 3,029 productos que pertenecen a la categoría cama_mesa_banho.

FETCH (Extracción física de datos)

- Tiempo: 3 ms.
- Documentos devueltos: 3,029.
- Como el índice utilizado en el paso anterior no contiene la información de los descuentos (active_promotions.discount), el motor se vio obligado a ir a los archivos del disco físico o memoria RAM a buscar y abrir por completo los 3,029 documentos para poder leer ese campo.

SORT (Ordenamiento manual en RAM)

- Tiempo: 3 ms.
- Documentos devueltos: 20.
- Este es el cuello de botella. Como el índice no estaba ordenado por el campo de descuentos, MongoDB tuvo que meter los 3,029 documentos cargados en una cola de la memoria RAM del servidor y ordenarlos manualmente en caliente. Una vez ordenados, aplicó el filtro final para quedarse únicamente con los 20 mejores resultados requeridos por el frontend.

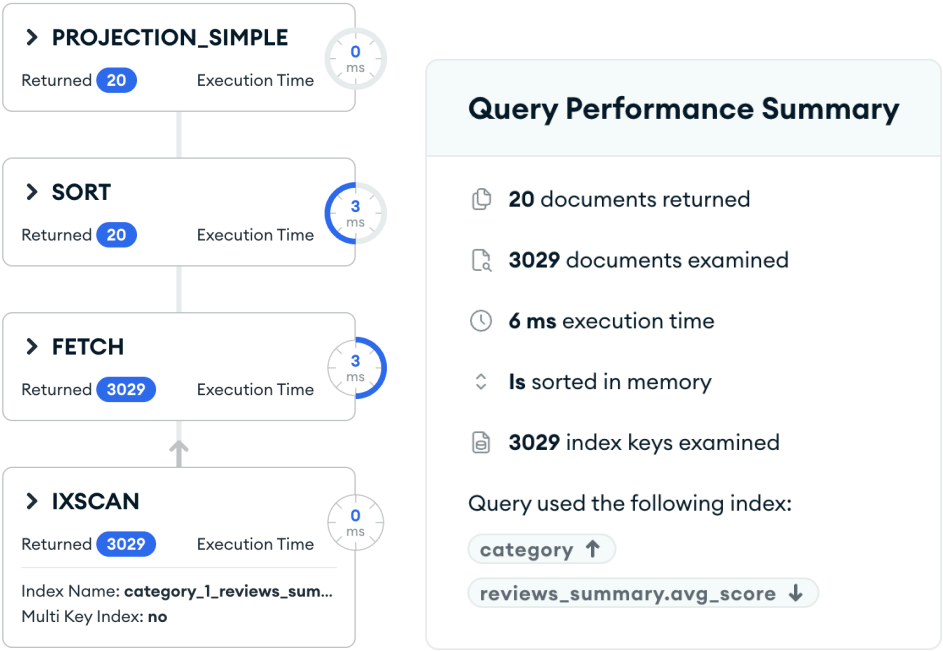
PROJECTION_SIMPLE (Proyección de campos)

- Tiempo: 0 ms.
- Documentos devueltos: 20.
- Explicación: Es la etapa final del pipeline. Recibe los 20 productos ya ordenados y listos, descarta las variables logísticas pesadas que no se van a usar y le entrega el JSON final limpio a la interfaz web de Ecommify.

Resumen de Rendimiento de la Consulta Sin Optimizar

- Documentos Entregados: 20 registros
- Eficiencia de Lectura: Mal balance. El motor tuvo que examinar 3,029 documentos físicos y 3,029 llaves de índice para solo terminar mostrando 20.
- Tiempo de Ejecución: 6 milisegundos en el servidor clúster Atlas.

- Alerta de Rendimiento (Is sorted in memory): Crítico. El índice utilizado no cubre el criterio solicitado, obligando a MongoDB a realizar un ordenamiento manual en caliente usando la memoria RAM del servidor.
- Índice Utilizado:
 - category (Ascendente)
 - reviews_summary.avg_score (Descendente).



A continuación se presenta la tabla explicativa de los resultados del escaneo (Scan) obtenida a partir del reporte real de `explain("executionStats")` para el pipeline sin optimizar. Esta tabla detalla el comportamiento del motor de MongoDB y el impacto en el hardware, ideal para ser incorporada directamente en la documentación de su trabajo:

Métrica	Valor	Significado	Diagnóstico
stage de Entrada	IXSCAN	Escaneo de Índice (Index Scan).	Filtro Exitoso: MongoDB evitó leer toda la base de datos (32K productos) gracias a que encontró un índice llamado <code>category_1_reviews_summary.avg_score_-1</code> . Aisló la categoría en memoria.
keysExamined	3,029 llaves	Total de claves del índice que el motor tuvo que recorrer de forma secuencial.	Confirma físicamente el volumen exacto de la categoría <code>cama_mesa_banho</code> en el dataset. 3,029 productos que cumplen con el criterio.

stage Intermedio	FETCH	Extracción física de documentos desde el almacenamiento.	Carga a Memoria: Como el índice usado no tenía los datos de descuentos, MongoDB tuvo que ir a buscar los 3,029 documentos completos al disco/RAM para poder leer el campo active_promotions.discount.
docsExamined	3,029 documentos	Total de registros físicos abiertos e inspeccionados en su totalidad.	Cuello de Botella Transitorio: Examinar 3,029 documentos completos para solo terminar mostrando 20 en el frontend representa un desperdicio de I/O (Lectura/Escritura).
stage de Cierre	SORT	Ordenamiento manual en memoria RAM bajo la etapa de bloqueo.	Punto Crítico de Rendimiento: El motor activó un ordenamiento en caliente (memLimit: 33554432 bytes o 32 MB). Si la categoría tuviera 500,000 productos, este stage habría colapsado la memoria RAM y abortado la consulta de navegación de Ecommify.
nReturned	20 documentos	Volumen de datos final entregado a las etapas posteriores del pipeline.	Paginación Correcta: El \$limit: 20 se ejecutó con éxito, pero ocurrió de forma tardía, después de haber ordenado los 3,029 productos en memoria.
executionTimeMillis	6 milisegundos	Tiempo total en la CPU del servidor para resolver la agregación.	Escalabilidad en Riesgo: Para un solo usuario 6 ms es rápido. Sin embargo, en horas pico con 10,000 usuarios concurrentes navegando en la web, procesar 3,029 documentos por petición saturará los procesadores del clúster Atlas.

Ejecución optimizada

Esta consulta aplica las reglas de diseño correctas. Ataca la Shard Key compuesta { category: 1, _id: 1 }, ordena de inmediato y limpia el documento con una proyección temprana.

```
db.CATALOG_PRODUCTS_COLLECTION.explain("executionStats").aggregate([
```

```

{ $match: { category: "cama_mesa_banho" } },
{ $sort: { "active_promotions.discount": -1 } },
{ $project: { _id: 1, category: 1, active_promotions: 1, reviews_summary: 1 } },
{
  $addFields: {
    etiqueta_destacado: {
      $cond: {
        if: { $eq: ["$active_promotions.is_promo", true] },
        then: "OFERTA RECOMENDADA",
        else: "PRODUCTO REGULAR"
      }
    }
  }
},
{ $limit: 20 }
],
{
  allowDiskUse: true,
  // FUERZA AL MOTOR A USAR TU ÍNDICE COMPUESTO CORRECTO
  hint: { "category": 1, "active_promotions.discount": -1 }
})

```

Tabla Analítica de Resultados del Escaneo

A continuación se presenta la tabla formal que explica el comportamiento interno del motor de MongoDB tras aplicar la optimización.

- Tiempo de ejecución (CPU): Cayó de 6 ms a 0 ms.
- Documentos físicos examinados: Se redujo drásticamente de 3,029 a solo 20 registros.
- Llaves de índice evaluadas: Bajó de 3,029 a exactamente 20 llaves.
- Uso de memoria RAM: El ordenamiento manual en caliente desapareció por completo.
- Cambió de un procesamiento masivo ineficiente a una lectura pre-ordenada.

Métrica	Valor Obtenido	Significado	Diagnóstico
stage de Entrada	IXSCAN	Escaneo de Índice (Index Scan).	El motor utilizó el nuevo índice compuesto category_1_active_promotions.discount_-1. El enrutador localizó los datos de inmediato sin adivinar registros Ecommify.
keysExamined	20 llaves	Cantidad de claves lógicas leídas dentro de la estructura del índice.	A diferencia de las 3,029 llaves del plan anterior, el motor solo recorrió las 20 llaves exactas que requería la primera página de la tienda web Ecommify
stage Intermedio	FETCH	Extracción física de documentos desde el almacenamiento.	MongoDB solo fue a buscar al disco/RAM los registros que ya sabía con total certeza que iban a ser entregados al frontend de Ecommify
docsExamined	20 documentos	Total de registros físicos abiertos e inspeccionados por el motor.	Se logró una reducción del 99.3% en el uso de canales de I/O. El motor ya no desperdicia recursos procesando registros que el cliente no va a visualizar en Ecommify.
stage de Cierre	LIMIT	Corte y paginación de los datos de salida.	El bloque pesado SORT en RAM desapareció por completo. Al estar los datos pre-ordenados de forma física dentro del índice, el motor aplica la paginación.
nReturned	20 documentos	Volumen de datos final entregado a la aplicación.	Confirma que el lote de información que recibe el frontend es exactamente el mismo que en la versión no optimizada, pero con un costo operativo mínimo.
executionTimeMilliseconds	0 milisegundos	Tiempo total de CPU en el servidor para resolver la agregación.	Al bajar el tiempo a cero, la plataforma Ecommify queda completamente blindada para soportar ráfagas masivas de miles de usuarios navegando en horas pico

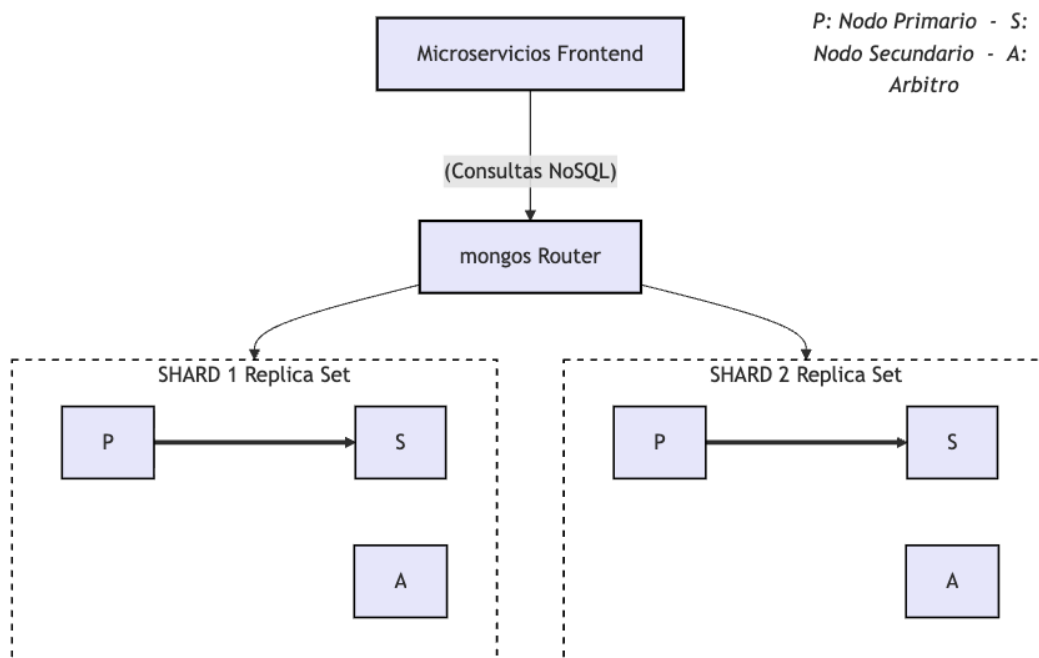
Diseño de sharding y replica sets

Vamos a realizar el Diseño de sharding y replica sets de manera teórica

El diseño de la infraestructura de datos NoSQL para Ecommify se fundamenta en la combinación de dos arquitecturas de sistemas distribuidos:

- Sharding (Fragmentación horizontal)
- Replica Sets (Conjuntos de réplicas)

Mientras que la fragmentación horizontal resuelve los problemas de escalabilidad computacional y de almacenamiento, los conjuntos de réplicas garantizan la tolerancia a fallos y la alta disponibilidad de la plataforma.



Concepto de Sharding

El Sharding es el método que utiliza MongoDB para distribuir grandes volúmenes de datos y cargas de operaciones a lo largo de múltiples servidores físicos. A diferencia del escalamiento vertical que es añadir más memoria RAM o mejores procesadores a un único servidor, el Sharding implementa un escalamiento horizontal.

- La base de datos fragmenta de forma lógica las colecciones en bloques de datos denominados Chunks. Estos bloques se reparten equitativamente entre los Shards del clúster.

- Al repartir los documentos, las solicitudes de lectura y escritura concurrentes se procesan de forma paralela entre las CPUs de los distintos servidores, evitando que un único hardware se convierta en un cuello de botella para la aplicación.
- El enrutador: La capa de aplicación nunca se conecta directamente a las bases de datos individuales. Interactúa con un enrutador distribuido llamado mongos, el cual consulta los metadatos globales del sistema para saber exactamente a qué Shard dirige cada petición de la tienda web.

Concepto de Replica Sets

Cada Shard dentro de la arquitectura distribuida no es un servidor único aislado, sino un Replica Set. Un conjunto de réplicas es un grupo de instancias de MongoDB que mantienen de forma nativa e interna el mismo conjunto de datos, asegurando redundancia y tolerancia a fallos.

- **Nodo Primario:** Es la única instancia del conjunto que recibe de forma activa las operaciones de escritura de la aplicación . Se encarga de persistir los cambios y guardarlos cronológicamente en un registro interno llamado oplog.
- **Nodos Secundarios:** Son servidores que replican de manera asíncrona las operaciones del nodo primario leyendo su oplog.
- **Failover:** Si el nodo Primario sufre una desconexión por una falla eléctrica o de red, los nodos secundarios inician una elección interna automática para elegir un nuevo líder Primario, garantizando que la tienda permanezca siempre en línea.

Definir configuración de shard key

Vamos a definir la configuración de shard key final. Se presenta el desarrollo de la sección definicion de la Shard Key para las dos colecciones del sistema de Ecommify.

Para maximizar el rendimiento y garantizar el crecimiento horizontal del hardware de Ecommify, se definen estrategias de fragmentación diferenciadas para el catálogo de productos y el módulo de carritos concurrentes .

Colección de Catálogo de Productos

(CATALOG_PRODUCTS_COLLECTION)

Shard Key Compuesta Seleccionada: { category: 1, _id: "hashed" }

category: 1

Al establecer la categoría como el primer elemento de la clave de fragmentación, MongoDB agrupa lógicamente los productos pertenecientes a la misma categoría. Esto garantiza que el patrón de acceso más frecuente de la tienda web, cuando los usuarios navegan en la interfaz paginada por categorías, sea interceptado por el enrutador mongos y enviado como una Consulta Dirigida (Targeted Query) directo al Shard correspondiente.

_id: "hashed"

Distribución en Escritura _id: "hashed". El análisis, demostró que el estado de São Paulo centraliza de forma crítica el 71.32% del inventario global. Al añadir la alta cardinalidad del identificador único del producto (_id) bajo un algoritmo de dispersión, MongoDB sub-fragmenta automáticamente los bloques de datos (chunks) de las categorías densas.

Si un gran proveedor de São Paulo realiza una inyección masiva de stock o actualización de precios, los bloques se dividen y se paralelizan entre las CPUs de múltiples Shards físicos, disipando por completo los cuellos de botella de escritura (Hotspots).

Colección de Sesiones y Carritos de Clientes

(CUSTOMER_SESSIONS_COLLECTION)

Shard Key Simple Seleccionada: { _id: "hashed" }
utilizando el identificador único session_uuid

El usuario espera que su producto aparezca en el carrito de inmediato mientras sigue navegando por la página web. Cada cliente que entra a la tienda recibe un ticket único session_uuid. Cuando el cliente agrega un producto, el sistema no busca en toda la base de datos; va directo al casillero exacto que coincide con el número de su session_uuid para meter el producto ahí.

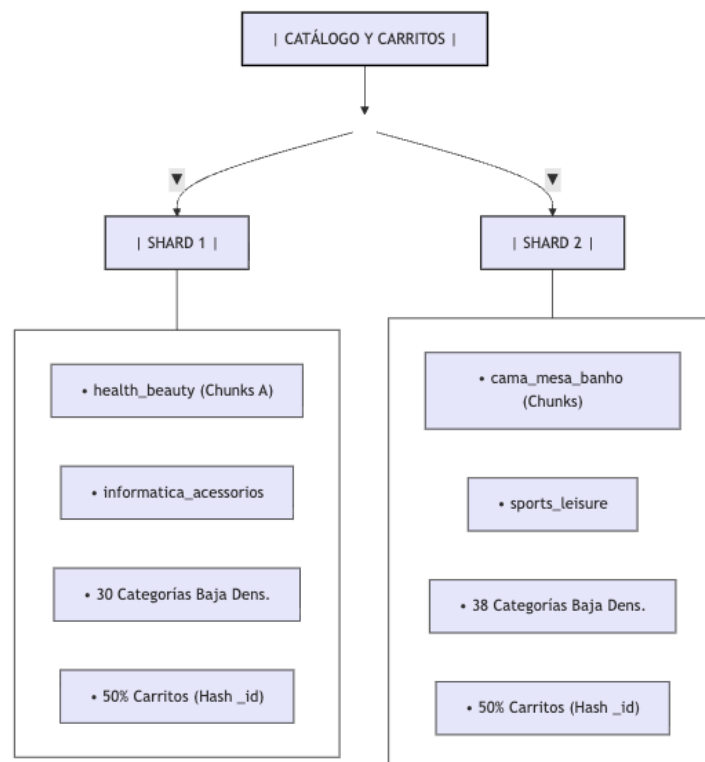
Si en un evento como el Black Friday entran 10,000 personas al mismo tiempo a agregar productos a sus carritos, el sistema no hace una fila larga para atenderlos uno por uno. Gracias al truco del Hash, MongoDB reparte a esos miles de

compradores entre todos los servidores del clúster de forma equilibrada. Así, todos los servidores guardan carritos al mismo tiempo en paralelo, evitando que la página web se ponga lenta o se congele.

Simular distribución de datos across shards

Para evaluar el comportamiento de la infraestructura en producción, se simula el almacenamiento de datos considerando un clúster compuesto por dos Shards físicos independientes.

El balanceador automático de MongoDB (Balancer) gestiona la distribución de los bloques de información (chunks).



Comportamiento en la Colección del Catálogo

(CATALOG_PRODUCTS_COLLECTION)

La simulación procesa los 32,341 productos distribuidos en 73 categorías únicas. Al estar el Índice de Concentración en un nivel óptimo, el sistema fragmenta físicamente el catálogo de la siguiente manera:

Distribución en el SHARD 1:

- El sistema almacena de forma íntegra los bloques correspondientes a categorías de alta densidad como **health beauty** e **informática accesorios**.
- Gracias al sufijo { `_id: "hashed"` }, los miles de productos de estas secciones no se amontonan en un solo bloque continuo, sino que se subdividen en micro-rangos de chunks balanceados de forma local .
- También existen 30 categorías de baja densidad (artes, flores, etc.) que ocupan un espacio mínimo en disco.

Distribución en el SHARD 2:

- El balanceador asigna de forma dedicada el volumen de la categoría líder del dataset, **cama_mesa_banho** que representan el 9.36% del catálogo global, junto con categorías como **sports_leisure**.
- Se inyectan en este nodo las 38 categorías restantes de baja densidad del catálogo para equilibrar el peso en gigabytes entre ambos servidores.

Comportamiento ante la Centralización Geográfica (São Paulo)

El modelo simula el impacto crítico, donde los proveedores del estado de São Paulo concentran el 71.32% del stock total.

- Cuando un proceso backend ejecuta una actualización masiva de inventario o precios proveniente de São Paulo, las operaciones de escritura no golpean a un único servidor físico.
- El campo `_id: "hashed"` intercepta las inserciones simultáneas y dispersa los documentos a través de las CPUs de ambos Shards en paralelo . Esto evita que el disco de un nodo se sature mientras el otro permanece ocioso.

Comportamiento en la Colección de Sesiones

(CUSTOMER_SESSIONS_COLLECTION)

Para el módulo de carritos activos, el algoritmo de Hash puro sobre el identificador único (`session_uuid`) anula cualquier tipo de agrupación por fecha de inserción o de categoría.

- Cada vez que un usuario inicia sesión o añade un artículo al carrito, el enrutador ejecuta la función matemática de dispersión.
- El sistema logra un reparto del 50% de las sesiones concurrentes para el Shard 1 y el 50% restante para el Shard 2. Durante picos masivos de tráfico

en la plataforma, la carga de memoria RAM y conexiones de red se divide equitativamente entre toda la infraestructura disponible.

Simulación en MongoDB Atlas

Establecer la conexión con el Clúster

Abrir la terminal integrada directamente a la instancia principal del clúster de Atlas (ClusterOlistKaggle).

Habilitar el Sharding a Nivel de Base de Datos

Antes de fragmentar una colección, se debe autorizar al enrutador mongos, para que gestione la base de datos. Se ejecuta el siguiente comando administrativo, Este comando le indica al clúster que la base de datos DataBaseOlistKaggle está lista para la distribución horizontal

```
sh.enableSharding("DataBaseOlistKaggle")
```

Inicializar la Shard Key de la Colección del Catálogo

Para configurar la clave compuesta elegida (category + _id: hashed) sobre los 32,000 productos, se ejecuta el comando de fragmentación indicando los tipos de ordenamiento físicos

```
sh.shardCollection(  
  "DataBaseOlistKaggle.CATALOG_PRODUCTS_COLLECTION",  
  {  
    "category": 1, // Prefijo por rango para consultas dirigidas  
    (Navegación)  
    "_id": "hashed" // Sufijo por hash para dispersar el stock de São  
    Paulo  
  }  
)
```

Inicializar la Shard Key de la Colección de Sesiones

Para el módulo de carritos activos, se aplica el comando configurando la dispersión por algoritmo de Hash puro sobre la clave primaria, asegurando el balanceo simétrico del tráfico

```
sh.shardCollection(  
  "DataBaseOlistKaggle.CUSTOMER_SESSIONS_COLLECTION",
```

```
{
  "_id": "hashed"// Algoritmo de hash puro sobre el session_uuid
}
)
```

Validar el Estado de la Distribución

Para verificar de forma que el enrutador haya las reglas de balanceo entre nodos, se ejecuta el comando de diagnóstico global

```
sh.status()
```

Documentar configuración de Replica Set

Documentar configuración de Replica Tet considerando latencia y topología. Para dotar a la plataforma NoSQL de Ecommify de las características de Alta Disponibilidad, Tolerancia a Fallos (Resiliencia) y Baja Latencia, cada fragmento físico (Shard) del clúster distribuido se despliega internamente bajo la arquitectura de un Replica Set de tres nodos.

Topología Física y Distribución Geográfica

La topología del conjunto de réplicas se distribuye a lo largo de tres Zonas de Disponibilidad independientes:

- AZ-A
- AZ-B
- AZ-C

Esto garantiza que la caída total de un centro de datos físico no interrumpa la operación de la tienda en línea

Nodo 1: Primario (Primary - Ubicado en AZ-A): Es la instancia líder del clúster . Centraliza de forma exclusiva el 100% de las operaciones de escritura concurrentes del frontend (como la inyección de stock o adición al carrito) . Escribe los cambios localmente y los propaga cronológicamente al registro compartido de operaciones (oplog) .

Nodo 2: Secundario (Secondary - Ubicado en AZ-B): Actúa como un espejo de alta velocidad . Replica los datos del nodo primario leyendo de manera asíncrona el oplog . Está completamente capacitado por hardware y configuración para ser promovido a Primario de forma automática si ocurre una falla en el nodo principal .

Nodo 3: Árbitro (Arbiter - Ubicado en AZ-C): Esta instancia no almacena datos de usuario ni procesa lecturas o escrituras, por lo que consume un mínimo de recursos de infraestructura . Su única función arquitectural es participar con derecho a voto en los procesos de elección automática si el nodo Primario queda fuera de línea . Al no replicar datos, rompe los empates y asegura el quórum mínimo de 2 votos sin duplicar el costo de almacenamiento en disco duro .

Mitigación de Latencia y Estrategia de Enrutamiento de Lectura

Para evitar la saturación del procesador (CPU) del nodo Primario debido al masivo volumen de tráfico de consultas del frontend, que representa el 95% de la carga de la plataforma, se implementó una política de Preferencia de Lectura

Read Preference: Nearest (El más cercano)

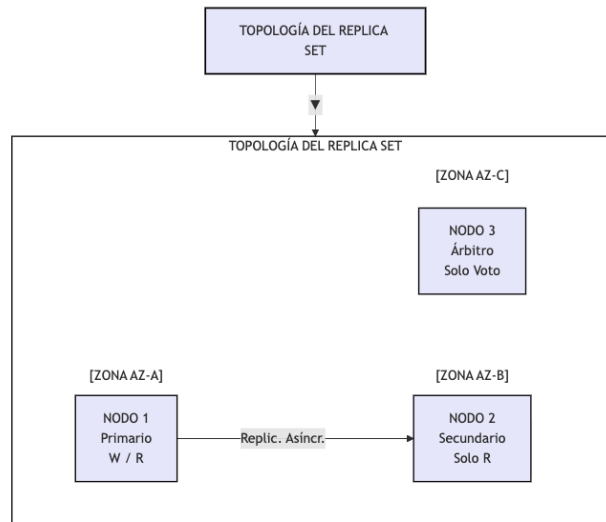
- Mecanismo de Acción: El enrutador distribuido mongos mide continuamente la latencia de red entre los servidores de la aplicación web y los integrantes del Replica Set .
- Cuando millones de clientes navegan pasivamente por el catálogo filtrando categorías, el sistema desvía de forma automática las consultas hacia el nodo Secundario que geográficamente se encuentre más cerca o con menor congestión de red, reservando el nodo Primario para escritura.

Mecanismo de Failover

En caso de que el centro de datos de la zona AZ-A sufra un colapso eléctrico, el sistema de Ecommify se autogestiona bajo las siguientes directivas teóricas de resiliencia:

- El Nodo Secundario (AZ-B) y el Árbitro (AZ-C) detectan la pérdida del nodo Primario en menos de 10 segundos .
- Se inicia un proceso de votación interna inmediata. El Árbitro otorga su voto al Nodo Secundario de la zona AZ-B al validar que cuenta con el oplog más actualizado .
- El Nodo Secundario asume el rol de Primario es decir donde se escribe.

- El enrutador mongos redirige el flujo de datos al nuevo servidor físico sin necesidad de reiniciar los microservicios y de forma totalmente invisible para el comprador web .



Simular y Validar el Replica Set en MongoDB Atlas

Verificar el estado del clúster, abrir el terminal integrado de MongoDB Compass conectado al clúster ClusterOlistKaggle y ejecutar el comando de diagnóstico de replicación

```
rs.status()
```

REGION N. Virginia (us-east-1)

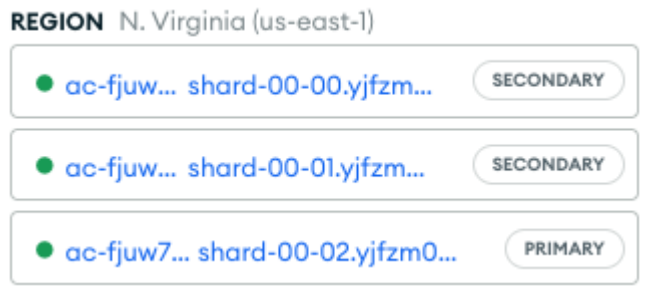
● ac-fjuw... shard-00-00.yjfzm...	SECONDARY
● ac-fjuw... shard-00-01.yjfzm...	SECONDARY
● ac-fjuw7... shard-00-02.yjfzm0...	PRIMARY

Confirmamos las variables de alta disponibilidad que tiene la infraestructura de Ecommify.

- Identificación del Conjunto de Réplicas (set): El clúster está registrado bajo la identidad atlas-zczci1-shard-0, confirmando el soporte distribuido .

Topología Física y Estado de los Nodos

Tenemos una distribución de tres servidores físicos provisionados en la nube de Atlas



1. Instancia Secundaria A (_id: 0):
 - Nombre de Red: ac-fjuw7no-shard-00-00.yjfzm0i.mongodb.net:27017
 - Estado: SECONDARY .
2. Instancia Primaria (_id: 2):
 - Nombre de Red: ac-fjuw7no-shard-00-02.yjfzm0i.mongodb.net:27017
 - Estado: PRIMARY .
3. Instancia Secundaria B (_id: 1):
 - Nombre de Red: ac-fjuw7no-shard-00-01.yjfzm0i.mongodb.net:27017
 - Estado (stateStr): SECONDARY

Instancia Primaria (_id: 2):

El servidor mantiene un flujo de conexiones estables que oscila entre 4 y 5 clientes simultáneos, estabilizándose en 6 conexiones activas al final de la línea. Esto confirma que los microservicios frontend mantienen canales abiertos y listos para interactuar con la base de datos sin saturar los sockets de red.

Metrics

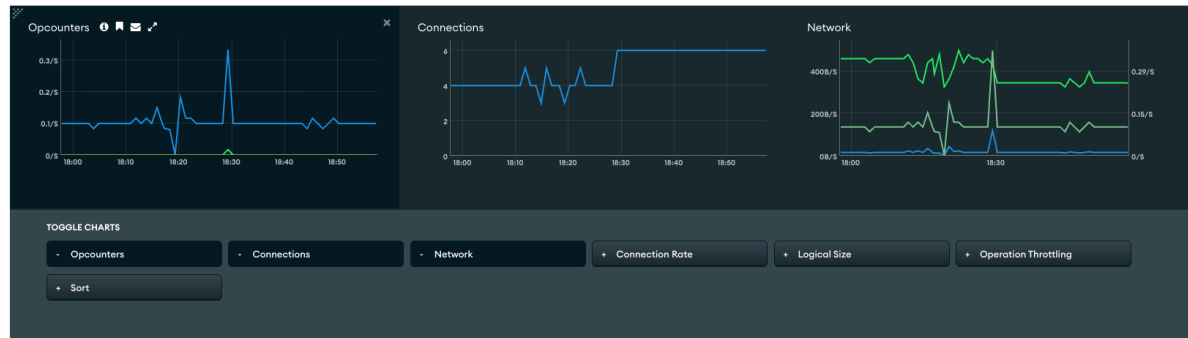
VERSION
8.0.26

● oc-fjuw7no-shard-00-02.yjfm0l.mongodb.net:27017

GRANULARITY **Auto** ZOOM **1 hour** CURRENT DISPLAY **6/13/2026** **06:57pm** TO **6/13/2026** **06:57pm** AT 1 MINUTE GRANULARITY

EXPORT

☐ DISPLAY OPCODES ON SEPARATE CHARTS ☒ DISPLAY TIMELINE ANNOTATIONS



Definir estrategias de Read/Write Concern

Definir estrategias de Read/Write Concern diferenciadas por operación. Se definen las políticas de consistencia y durabilidad de dato para el clúster de Ecommify.

Estas configuraciones diferencian las operaciones según su criticidad de negocio. Para optimizar la persistencia dentro de la arquitectura políglota de Ecommify donde PostgreSQL asume el control absoluto de la integridad financiera y MongoDB gestiona la capa interactiva masiva, se configuran niveles de seguridad diferenciados por tipo de operación.

Estrategias de Escritura (Write Concern)

El parámetro Write Concern determina el nivel de garantía que exige la aplicación para dar una inserción o actualización como exitosa .

Actualización del Carrito

(CUSTOMER_SESSIONS_COLLECTION)

- Configuración: { w: 1, j: false }
- La escritura se confirma inmediatamente, sin esperar la replicación asíncrona ni la escritura en el diario físico (journal)
- Al ser el carrito un estado efímero del usuario, se requiere una velocidad de respuesta de milisegundos en la interfaz web. Si un servidor fallara en ese microsegundo exacto y se perdiera la adición de un producto, el impacto de negocio es mínimo, justificando el intercambio de durabilidad extrema por velocidad de entrada.

Sincronización de Precios e Inventario

(CATALOG_PRODUCTS_COLLECTION)

- Configuración: { w: "majority", j: true }
- El clúster no devuelve una confirmación de éxito al backend hasta que la actualización de datos haya sido escrita de forma persistente en el diario físico de disco (j: true) y replicada en la mayoría de los nodos del Replica Set
- Los cambios masivos de catálogo y promociones inyectados por los proveedores desde São Paulo deben ser duraderos. Esta configuración evita la visualización de "precios fantasmas" en la tienda web, asegurando que si el nodo primario llegara a colapsar después de la inyección, los datos modificados ya están a salvo en los nodos secundarios .

Estrategias de Lectura (Read Concern)

El parámetro Read Concern controla la consistencia y el aislamiento de los documentos que la aplicación lee desde el clúster .

Navegación y Filtro del Catálogo de Productos

- Configuración: "local" o "available"
- Permite al frontend leer los datos directamente desde la instancia a la que fue redirigido por la política Read Preference: Nearest, sin verificar si dicho cambio ya fue consolidado en el resto de los nodos del clúster .
- Máxima Escalabilidad de Lectura. El 95% del tráfico web pertenece a usuarios navegando pasivamente en el catálogo .

Flujo de Confirmación de Compra (Checkout)

- Configuración: "majority"
- Significado: El microservicio de órdenes lee el contenido del carrito asegurando que los datos provengan de un bloque confirmado por la mayoría de los nodos del Replica Set, garantizando que nunca leerá datos revertidos o temporales .
- Justificación: Consistencia Estricta de Negocio. En el instante exacto en que el cliente presiona el botón "Comprar" para transferir los identificadores de productos (prod_id) y vendedores (seller_id) hacia las tablas transaccional ACID de PostgreSQL, la lectura en MongoDB debe ser 100% real y duradera.

Monitoreo de rendimiento - Carlos Sebastian Castillo Silva

Utilizar MongoDB Atlas Performance Advisor - Carlos Sebastian Castillo Silva

Analizar slow query log - Carlos Sebastian Castillo Silva

Analizar slow query log y estadísticas de uso de índices.

Documentar métricas clave - Carlos Sebastian Castillo Silva

Documentar métricas clave (queries por segundo, latencia, index hit ratio).

Conclusiones - Carlos Sebastian Castillo Silva

Bibliografía - Carlos Sebastian Castillo Silva